SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

ON

FOUNDATION COURSE
(TINKERING & ELEMENTS OF DESIGN)

SMART LOST & FOUND SYSTEM

Submitted in partial fulfillment of the requirement for the award of Degree of
Bachelor of Technology in Artificial Intelligence & Data Science

Submitted to:

Rajasthan Technical University, Kota (Raj.)

Submitted By:

Mitesh Nehra	24EARAD102
Mohit Kumar Sharma	24EARAD103
Naitik Pathak	24EARAD106
Paramveer Singh	24EARAD119
Prabhash Kumar Chaudhary	24EARAD121

Under the Supervision of
Er. Ram Babu Buri
Assistant Professor

Session:2026-27

Department of Artificial Intelligence & Data Science
ARYA COLLEGE OF ENGINEERING & I.T.
SP-42, RIICO INDUSTRIALAREA, KUKAS, JAIPUR

Table of Contents

1. Introduction 1
1.1 Purpose 1
1.2 Scope 1
1.3 Definitions and abbreviations 1
1.4 Reference 2
1.5 Overview 2
2. Overall Description 3
2.1 Product perspective 3
2.2 Product functions 3
2.3 User classes 4
2.4 Operating environment 4
2.5 Design and implementation constraints 4
2.6 Assumptions and dependencies 4
3. Specific Requirements 5
3.1 External interface requirements 5
3.2 Functional requirements 5
3.3 Matching rule (FR-14) 7
3.4 Non-functional requirements 8
4. Data Requirements 9
4.1 Logical data model 9
4.2 Status model 9
5. Constraints, Known Gaps and Acceptance 10
5.1 Known gaps in the current build 10
5.2 Acceptance criteria 11
5.3 Traceability to test cases 12

1. Introduction

1.1 Purpose

This document specifies the software requirements of the Smart Lost & Found System, a desktop application for reporting, reviewing and claiming lost and found belongings on a college campus. It is written for the project team, the mentor and the evaluators, and follows the structure of IEEE Std 830 / ISO/IEC/IEEE 29148. It describes what the system does and the constraints under which it works. Design details are covered in the project report.

1.2 Scope

The system gives the campus a single, centralized record of lost and found items. Students register, log in, report lost and found items and follow their status. Administrators verify reports, register items handed over at the campus desk and view claimed items and registered users. Data is stored in a MySQL database accessed through JDBC. The system is a stand-alone Java Swing application for Windows. It does not include a web or mobile interface, image upload, or e-mail/SMS delivery.

1.3 Definitions and abbreviations

Term	Meaning
DAO	Data Access Object, a class that holds all SQL for one entity
JDBC	Java Database Connectivity

RBAC	Role-based access control
Student	A registered user who reports and claims items
Admin	The campus administrator who verifies reports and records
Lost item / Found item	A report about a missing item / an item that someone has found
Claim	A student's request to receive an approved item

1.4 References

1. IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications.
2. Oracle, The Java Tutorials (Swing, JDBC).
3. MySQL 8.0 Reference Manual and MySQL Connector/J Developer Guide.
4. Project source code (SmartLostAndFound/src) and database script (database/database.sql).

1.5 Overview

Section 2 gives the general description, Section 3 lists the specific requirements, Section 4 describes the data and status model, and Section 5 lists constraints, known gaps and acceptance criteria.

2. Overall Description

2.1 Product perspective

The product is a self-contained client application. It follows a layered architecture: Swing user interface (LoginFrame, RegisterDialog, DashboardFrame, UITheme), model classes (User, LostItem, FoundItem, Claim), a service class (MatchingService), DAO classes (UserDAO, LostItemDAO, FoundItemDAO, ClaimDAO) and a singleton DatabaseConnection that talks to MySQL. Database settings are read from a db.properties file, and the tables are created on first launch if they are missing.

2.2 Product functions

- Student registration and login with a role selector.
- Reporting of lost items and found items with item details and contact number.
- Administrative review: approve or reject lost and found reports.
- In-app notification to the student when a report is approved, followed by the student's accept/claim or return action.
- Desk entry of found items by the administrator.
- Lists of registered users and claimed items for the administrator, and of claimed items for the student.
- Rule-based comparison of a lost item with found items (MatchingService)

2.3 User classes

User class	Characteristics	Main permissions
Student	College student, basic computer skills, registers by self	Register, log in, report lost/found items, view own reports and approvals, accept or return an approved item, view claimed items
Administrator	Campus staff, single predefined account	Log in, view registered users, register desk found items, approve or reject lost and found reports, view claimed items

2.4 Operating environment

Windows 10/11, JDK 17 (for building; the packaged JAR/EXE is used to run), MySQL Server 8.x on localhost, MySQL Connector/J on the classpath, display of at least 1366 x 768, dual-core CPU and 4 GB RAM.

2.5 Design and implementation constraints

The application is written in Core Java with Swing. All database access goes through DAO classes using JDBC, and user input must reach SQL only through PreparedStatement parameters. The project must be completed within the 12-week plan (6 July to 6 October 2026) by a team of five.

2.6 Assumptions and dependencies

MySQL is installed, running and reachable with the credentials in db.properties. Users have a valid college e-mail address and a 10-digit mobile number. A single campus administrator manages verification. The application is used from a campus computer, not over the Internet.

3. Specific Requirements

3.1 External interface requirements

User interface :

A login window with role selection (STUDENT or ADMIN), e-mail, password, show-password option and a registration button. A registration dialog for students. After login a dashboard with a left navigation bar and content panels. The student dashboard has four panels: Report Lost Item, Report Found Item, Notifications & Approvals and My Claimed Items. The administrator dashboard has four panels: Registered Users Details, Register Found Items, Lost Items Approval and Claimed Items Details. Status values are shown in tables with colour-coded text, and every dashboard has a Logout button.

Software interface :

JDBC connection to MySQL using MySQL Connector/J. Connection parameters (host, port, database name, user, password) are read from db.properties.

Hardware interface :

Standard keyboard, mouse and display. No special hardware.

3.2 Functional requirements

ID	Requirement	Priority
FR-01	The system shall let a student register with full name, college roll number/ID, 10-digit mobile number, college e-mail, password and password confirmation.	High

FR-02	The system shall reject registration if any field is empty, the mobile number is not exactly 10 digits, the e-mail is malformed, the passwords differ, or the e-mail is already registered.	High
FR-03	The system shall authenticate a user by e-mail and password and open the dashboard that matches the user's role. If the selected role differs from the account role, the system shall show a message asking for the correct role.	High
FR-04	A student shall be able to submit a lost-item report with item name, category, campus location, lost date (YYYY-MM-DD), contact mobile number and an optional description.	High
FR-05	A student shall be able to submit a found-item report with item name, category, found location, found date, contact mobile number and description/storage details.	High
FR-06	The system shall validate required fields, the date format and the 10-digit mobile number before saving a report, and shall show a clear message on failure.	High
FR-07	A newly saved student report shall have the status PENDING.	High
FR-08	The administrator shall be able to view pending lost-item reports and found-item reports with reporter name and contact number, and approve or reject each one.	High
FR-09	When a lost-item report is approved, the system shall notify the student in the application and let the student accept and claim the item or return it to the administrator.	High
FR-10	The administrator shall be able to register a found item handed over at the campus desk (item name, category, found location, desk storage location, date, finder name, contact phone).	Medium

FR-11	The system shall show the administrator a list of all registered users with ID, name, roll number, e-mail and mobile number.	Medium
FR-12	The system shall show claimed items to the student (own items) and to the administrator (all items, with claimant name, roll number and mobile number).	Medium
FR-13	The system shall provide a refresh action and a status filter on the list screens.	Low
FR-14	The system shall provide a rule-based comparison that, for a lost item and a found item, decides whether they are a possible match (see 3.3).	Medium
FR-15	The system shall store every user, report and claim persistently in MySQL, and shall create the required tables on first run if they do not exist.	High
FR-16	A user shall be able to log out, which closes the dashboard and returns to the login window.	High

3.3 Matching rule (FR-14)

The MatchingService decides that a lost item and a found item are a possible match when both of the following hold:

1. The categories are equal, ignoring case and surrounding spaces.
2. The item names are not empty and one name contains the other, or some word of the lost item's name that is at least 3 characters long occurs in the found item's name (case-insensitive).

The service offers findMatchesForLostItem and findMatchesForFoundItem, which return the list of matching records, and getMatchLabel, which returns "Possible Match Found" or "No Match Found". A match is only a suggestion and is never treated as proof of ownership. Final verification is done by the administrator.

3.4 Non-functional requirements

Attribute	Requirement
Usability	Forms use labelled fields, required fields are marked with an asterisk, and error messages explain what to correct.
Performance	Opening a screen or refreshing a list shall complete within about 2 seconds for a campus-sized data set (up to a few thousand records) on the minimum hardware.
Reliability	A database error shall produce an error message and shall not terminate the application. A warning is shown at start-up if the database is unreachable.
Security	All SQL that contains user input shall use PreparedStatement. Administrative screens shall be reachable only after logging in with the ADMIN role. Database credentials shall be kept in db.properties and not in public repositories.
Maintainability	UI, model, service and DAO responsibilities shall stay in separate packages.
Portability	The application shall run on Windows 10/11 with JDK 17 or the packaged EXE/JAR.
Data integrity	Foreign keys with InnoDB shall link reports and claims to users; e-mail shall be unique.

4. Data Requirements

4.1 Logical data model

Table	Main columns	Notes
users	id (PK), full_name, email (unique), password, role (STUDENT/ADMIN), phone, college_id, created_at	One administrator account is predefined
lost_items	id (PK), user_id (FK), item_name, category, location, lost_date, description, contact_phone, status, created_at	Deleted with its user (cascade)
found_items	id (PK), user_id (FK), item_name, category, found_location, storage_location, found_date, description, contact_phone, status, created_at	Deleted with its user (cascade)
claims	id (PK), lost_item_id (FK, nullable), found_item_id (FK), claimant_id (FK), proof_details, claim_status, admin_remarks, created_at, updated_at	Kept for claim records and compatibility

4.2 Status model

Item reports use the values PENDING, APPROVED, REJECTED, ACTIVE, CLAIMED, RETURNED and RESOLVED. Claim records use PENDING, APPROVED, REJECTED and CONFIRMED.

From	Event	To
(new report)	Student submits	PENDING
PENDING	Admin approves	APPROVED
PENDING	Admin rejects	REJECTED
APPROVED	Student accepts and claims the item	CLAIMED
CLAIMED	Student confirms receipt (claim flow)	RETURNED (found) / RESOLVED (lost)
Claim PENDING	Admin approves or rejects	APPROVED / REJECTED

5. Constraints, Known Gaps and Acceptance

5.1 Known gaps in the current build

These points were found by reading the submitted source code and should be fixed or stated openly in the report.

1. **Passwords are stored and compared as plain text** in the users table. Hashing (for example BCrypt or salted SHA-256) is required before real use.
2. **The database script and start-up code insert a fixed administrator and five fixed student accounts** with known passwords. These should be removed for deployment, or the report should state that they are demo accounts.

3. **MatchingService is not called from the user interface.** The class and the search methods in the DAOs exist, but the dashboard only creates the object. Matches are therefore not displayed yet.
4. **The matching rule is a yes/no rule, not a weighted 0-100 score.** The score weights (30/20/25/25) in the README and the report are not implemented in MatchingService.java.
5. **The student interface has no proof-details field.** Claims are made with an Accept & Claim action on an approved item, although the claims table has a proof_details column.
6. **Roles are enforced by building a different dashboard for each role.** The DAO methods do not check the caller's role.
7. **There is no multi-statement transaction control** (setAutoCommit/commit/rollback) in the DAO classes, so a status change that updates two tables is not atomic.
8. **Notifications are in-app only.** There are no e-mail or SMS messages and no audit log.

5.2 Acceptance criteria

The system is accepted when: a student can register and log in; a student report is saved as PENDING and appears in the administrator's list; the administrator can approve or reject it; an approved report appears in the student's Notifications & Approvals panel and can be accepted, after which it appears under Claimed Items for both student and administrator; invalid input (empty field, bad date, wrong phone, duplicate e-mail, wrong password) is rejected with a message; data is still present after the application is closed and reopened; and logging out returns to the login window.

5.3 Traceability to test cases

Requirement	Test case	Coverage
FR-01	Register with valid details	Covered
FR-02	Register with duplicate e-mail	Partly: add cases for empty field, bad phone, bad e-mail and unequal passwords
FR-03	Login with valid student credentials; login with valid administrator credentials; login with invalid password	Covered; add a wrong-role case
FR-04	Submit lost-item form with required fields	Covered
FR-05	Submit a found-item report	Covered
FR-06	Submit form with missing required fields	Covered; add bad date and bad phone
FR-07	Submit lost-item form; submit a found-item report	Covered (check that status is PENDING)
FR-08	Admin updates a report status	Covered
FR-09	Admin updates a report status	Partly: notification and accept action need their own case
FR-10	None	Add a test for desk entry of a found item
FR-11	None	Add a test for the registered users list

FR-12	None	Add a test for the claimed items lists
FR-13	Search by item keyword, category or location	Covered (filter and refresh)
FR-14	Run possible-match search	Cannot pass until MatchingService is called from the interface (see 5.1, point 3)
FR-15	Launch application with database available; close and reopen application	Covered
FR-16	Logout	Covered
Security (3.4)	Enter SQL-like text in a form	Covered: SQL-like input is treated as data